Post-Quantum Privacy for Decentralized Governance: ML-KEM-768 Handshake and PQC-Encrypted Mail

Tristan Stoltz
Luminous Dynamics
tristan.stoltz@evolvingresonantcocreationism.com

March 2026

Abstract

We present a post-quantum privacy architecture for decentralized governance systems, implemented across two Mycelix components: the Identity cluster's hybrid key infrastructure (Ed25519 + ML-KEM-768) and the Mail cluster's 12-zome PQC-encrypted email system. The architecture provides three layers of quantum resistance: (1) a hybrid handshake protocol combining classical Diffie-Hellman with ML-KEM-768 key encapsulation, ensuring security even if one scheme is broken; (2) AES-256-GCM message encryption using keys derived from the hybrid handshake; and (3) forward secrecy through ephemeral key pairs per conversation. We address the practical challenges of deploying PQC in a decentralized context: larger key sizes (1,184 bytes for ML-KEM-768 vs. 32 bytes for Ed25519), DHT bandwidth implications, and backward compatibility with agents that have not yet upgraded to post-quantum keys. The system is integrated with consciousness-gated access control, where encryption key exchange requires Participant+ tier to prevent anonymous spam.

1 Introduction

The advent of cryptographically relevant quantum computers (CRQCs) threatens the security foundations of all current decentralized systems. RSA and elliptic-curve cryptography, which underpin blockchain signatures, TLS connections, and identity verification, are vulnerable to Shor's algorithm [1].

For governance systems, the quantum threat is particularly severe:

1. **Identity forgery**: An attacker with a CRQC could forge agent signatures, impersonating legitimate governance participants.
2. **Historical decryption**: Messages encrypted with classical key exchange can be stored now and decrypted later when CRQCs become available ("harvest now, decrypt later").
3. **Credential forgery**: Consciousness credentials signed with classical keys become forgeable, undermining the entire governance tier system.

NIST's post-quantum cryptography standardization [2] provides the building blocks for quantum-resistant systems. We apply these to Mycelix's governance infrastructure.

2 Threat Model

We consider an adversary with the following capabilities:

- Access to a future CRQC capable of running Shor's algorithm
- Ability to record all DHT traffic ("harvest now")
- Participation in the Holochain network as a valid agent
- Cannot compromise a majority of DHT validators simultaneously

Our security goals:

1. **Confidentiality**: Messages remain private even against a future CRQC.

2. **Integrity**: Source chain entries and credentials cannot be forged.
3. **Forward secrecy**: Compromise of long-term keys does not reveal past communications.
4. **Backward compatibility**: Agents without PQ keys can still participate (with reduced security guarantees).

3 Hybrid Key Infrastructure

3.1 DID Document Keys

Each Mycelix DID document contains two key types:

Authentication Ed25519 (32-byte public key). Used for source chain signing, DHT validation, and Holochain's native operations.

Key Agreement ML-KEM-768 (1,184-byte public key). Used for encrypted communication, PQC mail, and future signature verification.

3.2 ML-KEM-768 Properties

ML-KEM-768 (FIPS 203) [2] is a lattice-based key encapsulation mechanism:

Parameter	Value
Security level	NIST Category 3 (AES-192)
Public key	1,184 bytes
Secret key	2,400 bytes
Ciphertext	1,088 bytes
Shared secret	32 bytes
Encapsulation	< 1 ms
Decapsulation	< 1 ms

Table 1: ML-KEM-768 parameters.

3.3 Key Size Impact on DHT

The 37× increase in public key size (1,184 vs. 32 bytes) has measurable but manageable impact on DHT operations:

- **Storage**: DID documents grow by ∼1.2 KB per agent. With 3× DHT replication, each agent's PQ key adds ∼3.6 KB to the network.
- **Bandwidth**: DID document retrieval (for key exchange) adds ∼1.2 KB per query. This is negligible compared to typical entry sizes (1–10 KB).
- **Gossip**: DHT gossip propagating DID updates transmits the full document. PQ keys add ∼1.2 KB per gossip message for DID updates specifically.

For a network of 10,000 agents, the total PQ key storage overhead is ∼36 MB (with replication)—negligible for modern hardware.

4 Hybrid Handshake Protocol

4.1 Protocol Description

The hybrid handshake combines classical X25519 Diffie-Hellman with ML-KEM-768 key encapsulation:

Algorithm 1 Hybrid Handshake

Initiator A, **Responder** B
A: Generate ephemeral X25519 key pair (ek_A, EK_A)
A: Retrieve B's ML-KEM-768 public key PK_B^{PQ} from DHT
A: $(ct, ss_{\mathrm{PQ}}) \leftarrow \text{ML-KEM.Encaps}(PK_B^{\mathrm{PQ}})$
$A \rightarrow B$: $\{EK_A, ct\}$
B: Generate ephemeral X25519 key pair (ek_B, EK_B)
B: $ss_{\mathrm{classical}} \leftarrow \text{X25519}(ek_B, EK_A)$
B: $ss_{\mathrm{PQ}} \leftarrow \text{ML-KEM.Decaps}(sk_B^{\mathrm{PQ}}, ct)$
$B \rightarrow A$: $\{EK_B\}$
A: $ss_{\mathrm{classical}} \leftarrow \text{X25519}(ek_A, EK_B)$
Both: $K \leftarrow \text{HKDF}(ss_{\mathrm{classical}} \| ss_{\mathrm{PQ}})$

The final session key K is derived from both shared secrets using HKDF. This provides IND-CCA2 security if *either* X25519 or ML-KEM-768 is secure.

Theorem 1 (Hybrid security)**.** *If either X25519 (under the CDH assumption) or ML-KEM-768 (under the Module-LWE assumption) is IND-CCA2 secure, then the hybrid handshake pro-*

duces a session key indistinguishable from random.

This follows from the standard hybrid argument: an adversary that can distinguish the session key from random must break *both* underlying schemes.

4.2 Forward Secrecy

Forward secrecy is provided by the ephemeral X25519 key pairs. Even if long-term ML-KEM-768 keys are compromised, past session keys cannot be recovered because the ephemeral X25519 keys are erased after key derivation.

4.3 Fallback for Non-PQ Agents

Agents without ML-KEM-768 keys (not yet upgraded) fall back to classical-only X25519 key exchange. The system maintains backward compatibility while encouraging upgrade through:

- Visual indicators showing PQ status in the UI
- Consciousness profile bonus for PQ key registration
- Governance recommendations to require PQ keys for Steward+ tier

5 PQC-Encrypted Mail

5.1 12-Zome Architecture

The Mail cluster implements decentralized email with 12 zomes:

1. **compose**: Message composition and encryption
2. **inbox**: Incoming message management
3. **outbox**: Sent message tracking
4. **folders**: Custom folder management
5. **contacts**: Contact list with PQ key caching
6. **attachments**: Encrypted attachment storage
7. **threads**: Conversation threading
8. **search**: Encrypted search over local messages
9. **spam**: Consciousness-gated spam filtering
10. **keys**: Key management and rotation
11. **notifications**: Push notification handling
12. **bridge**: Cross-cluster mail dispatch

5.2 Message Encryption

Each message is encrypted using AES-256-GCM with a key derived from the hybrid handshake:

1. Initiator performs hybrid handshake with recipient (Section 4)
2. Session key K is derived via HKDF
3. Message plaintext is encrypted: $C = \text{AES-256-GCM}(K, \text{nonce}, M)$
4. Encrypted message is stored on sender's source chain (outbox) and sent to recipient's DHT neighborhood (inbox)
5. Recipient decapsulates their ML-KEM-768 ciphertext to recover K and decrypts the message

5.3 Conversation Forward Secrecy

Each conversation uses a ratcheting key derivation:

$$K_{n+1} = \text{HKDF}(K_n \| \text{message_hash}_n) \quad (1)$$

This ensures that compromise of key K_n does not reveal messages encrypted with K_{n-1} or earlier, providing conversation-level forward secrecy.

5.4 Spam Prevention

Mail sending requires Participant+ consciousness tier. This is the most effective decentralized spam prevention: creating a Participant-tier identity requires non-trivial identity verification ($I \geq 0.25$) and community engagement, making spam account creation expensive.

Additionally, the spam zome implements:

- Rate limiting per sender (configurable per recipient)
- Reputation-weighted spam scoring (low-reputation senders are flagged)
- Community blocklists (shared among trusted peers)

6 Reputation System Security

6.1 Temporal Decay

Reputation scores decay exponentially:

$$R(t) = R_0 \cdot 0.998^{(t-t_0)/\text{day}} \tag{2}$$

This prevents an agent from accumulating permanent reputation from past good behavior and then exploiting it. The half-life of $\frac{\ln 2}{\ln(1/0.998)} \approx 346$ days means reputation approximately halves yearly.

6.2 Slashing

Severe misbehavior (detected by safety agents or community reports) triggers reputation slashing:

$$R \leftarrow 0.5 \cdot R \tag{3}$$

Multiple slashing events compound: three slashing events reduce reputation to $0.125 \cdot R_0$, pushing most agents below the blacklist threshold of 0.05.

6.3 Blacklisting and Restoration

Agents with $R < 0.05$ are blacklisted from all governance participation. Restoration requires 100 verified positive interactions—approximately 3–4 months of active, positive community participation.

7 Implementation Considerations

7.1 PQC Library Selection

We use the `ml-kem` Rust crate (pure Rust implementation of FIPS 203). Key considerations:

- **WASM compatibility**: The crate compiles to `wasm32-unknown-unknown` for Holochain zomes.
- **Constant-time operations**: The implementation uses constant-time comparisons to prevent timing side-channels.
- **No system dependencies**: Pure Rust avoids C library linking issues in the WASM target.

7.2 Key Storage

ML-KEM-768 secret keys (2,400 bytes) are stored in the agent's secure keystore, managed by Holochain's conductor. The conductor's keystore provides:

- Encrypted at-rest storage
- Memory protection (mlock where available)
- Key derivation from a master secret

8 Evaluation

8.1 Performance

ML-KEM-768 encapsulation/decapsulation adds < 2 ms to message send/receive operations. This is negligible compared to DHT propagation latency (typically 100–500 ms).

8.2 Bandwidth Overhead

Per-message overhead from PQC:

- ML-KEM-768 ciphertext: 1,088 bytes (first message only, then ratcheted)
- AES-256-GCM: 16-byte tag + 12-byte nonce = 28 bytes per message
- Total first-message overhead: ∼1.1 KB
- Subsequent message overhead: 28 bytes

8.3 Migration Path

The hybrid architecture supports incremental migration:

1. **Phase 1** (current): Classical keys required, PQ keys optional. Hybrid handshake used when both parties have PQ keys.
2. **Phase 2** (planned): PQ keys recommended for Citizen+ tier. Governance actions may require PQ keys.
3. **Phase 3** (future): PQ keys required for all agents. Classical keys retained for backward compatibility only.

9 Related Work

Signal Protocol [3]: Provides forward secrecy and ratcheting but uses classical-only key exchange. The PQXDH proposal [4] adds post-quantum keys but is centralized (Signal servers).

Matrix/Olm [5]: Federated encrypted messaging with classical-only keys. No PQC migration plan published.

ProtonMail: Server-mediated PQC encryption. Not decentralized.

Our contribution is the first PQC-encrypted messaging system that is (1) fully decentralized (Holochain DHT, no servers), (2) hybrid (classical + PQ), and (3) integrated with consciousness-gated access control.

10 Conclusion

Post-quantum privacy for decentralized governance is both necessary and practical. The hybrid handshake protocol provides quantum resistance today without sacrificing interoperability with classical infrastructure. The 12-zome PQC-encrypted mail system demonstrates that full-featured decentralized email is achievable with acceptable performance overhead (< 2 ms per operation, $\sim$1.1 KB first-message bandwidth). The integration with consciousness gating (Participant+ for mail, Citizen+ recommended for PQ keys) provides Sybil-resistant spam prevention while encouraging adoption of quantum-resistant cryptography.

References

[1] Shor, P. W. (1994). Algorithms for Quantum Computation: Discrete Logarithms and Factoring. *FOCS*.

[2] NIST (2024). Module-Lattice-Based Key-Encapsulation Mechanism Standard. FIPS 203 (ML-KEM).

[3] Marlinspike, M. and Perrin, T. (2016). The X3DH Key Agreement Protocol. Signal Specification.

[4] Signal (2023). PQXDH: Post-Quantum Extended Diffie-Hellman. Signal Specification.

[5] Matrix.org Foundation (2020). Olm: An Implementation of the Double Ratchet. Matrix Specification.

[6] Brock, A. and Harris-Braun, E. (2018). Holochain: Scalable Agent-Centric Distributed Computing.